

CS 305: THEORY OF COMPUTATION — WEEK 3 LECTURE

Nondeterministic Finite Automata (NFA)

Parallel Computation, ϵ -Transitions, Powerset Construction & Search Engines

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Textbook Reference: Michael Sipser, *Introduction to the Theory of Computation* (3rd Ed.), Section 1.2

Week 2 Recap & Week 3 Roadmap

Moving from rigid deterministic transitions to multi-branching parallel computation

Week 2 Foundations Recap

What We Established (DFAs):

- **Strict Determinism:** Exactly one outgoing transition per symbol: $\delta : Q \times \Sigma \rightarrow Q$.
- **Total Functions:** No missing edges, no ambiguous choices, and no spontaneous jumps.
- **Product Construction:** Cartesian state space $Q_1 \times Q_2$ proves closure under Union ($\cup$) and Intersection ($\cap$).
- **System Speed:** $\mathcal{O}(1)$ throughput per byte in 100 Gbps network firewalls (Snort/Zeek).

Today's Agenda (Week 3)

Core Topics Today:

- **The Nondeterminism Paradigm:** Multiple choices and spontaneous jumps (ϵ -transitions).
- **Formal NFA 5-Tuple:** Transition function to powersets $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$.
- **Computation Trees:** Existential acceptance ($\exists$) vs. Universal rejection ($\forall$).
- **The Rabin–Scott Theorem:** Proving $\mathcal{L}(\text{NFA}) \equiv \mathcal{L}(\text{DFA})$ via Powerset Construction.
- **Closure via NFAs:** Concatenation ($A \circ B$) and Star (A^*).
- **Systems Spotlight:** Ken Thompson's $\mathcal{O}(m \cdot n)$ NFA simulation in grep and search engines.

Pedagogical Milestone: Problem Set 1 is Due Today. Today we unlock NFA techniques required for regular operations and regex engines.

Why Nondeterminism? The Power of Parallel Hypotheses

Overcoming state explosion when patterns require “looking into the future”

The DFA Design Dilemma

The Suffix Challenge:

Consider designing a machine for:

$$L = \{w \in \{0, 1\}^* \mid \text{the 3rd symbol from the end is } 1\}$$

Why DFAs Struggle:

- A DFA cannot know when the string will end.
- It must maintain memory of all $2^3 = 8$ possible 3-bit suffixes at every step!
- For the k -th symbol from the end, a DFA requires 2^k states.

Core Takeaway: Nondeterminism dramatically simplifies automaton design and enables modular proofs, without adding physical hardware requirements.

The NFA Superpower (4 States!)



Two Equivalent Mental Models:

1. **Parallel Universe Forking:** On reading ‘1’ in q_0 , the machine forks: one branch stays in q_0 , the other guesses this ‘1’ starts the final 3-bit suffix.
2. **Lucky Oracle:** The machine always makes the optimal guess that leads to acceptance.

Formal Definition of an NFA

Michael Sipser, Definition 1.16: Mathematical 5-tuple allowing sets of states and ϵ

The NFA 5-Tuple Specification

Definition 3.1: NFA 5-Tuple

A **Nondeterministic Finite Automaton (NFA)** is a 5-tuple:

$$N = (Q, \Sigma, \delta, q_0, F)$$

where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ is the transition function.
4. $q_0 \in Q$ is the start (initial) state.
5. $F \subseteq Q$ is the set of accept (final) states.

Notational Definitions:

- $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ (alphabet including empty string).
- $\mathcal{P}(Q) = 2^Q = \{R \mid R \subseteq Q\}$ (the power set of Q).

DFA vs. NFA Comparison

Architectural Differences:

Property	DFA	NFA
$\delta(q, a)$ Output	Single state $p \in Q$	Set $R \subseteq Q$
Input Domain	$Q \times \Sigma$	$Q \times \Sigma_\epsilon$
ϵ -Transitions	Forbidden	Permitted
Missing Edges	Illegal (total)	Permitted ($\emptyset$)
Active Paths	Exactly 1	Multiple parallel

Empty Set Transitions ($\delta = \emptyset$):

If $\delta(q, a) = \emptyset$, that specific computation branch **dies immediately** (silent rejection). Other parallel branches continue.

Formal Computation & Acceptance Criteria

Michael Sipser, Definition 1.18: Existential acceptance over parallel computation branches

Formal Acceptance Definition

Definition 3.2: NFA String Acceptance

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. Let string $w \in \Sigma^*$.

N **accepts** w if w can be written as:

$$w = y_1 y_2 \dots y_m$$

where each $y_i \in \Sigma_\epsilon = \Sigma \cup \{\epsilon\}$, and there exists a state sequence $r_0, r_1, \dots, r_m \in Q$ such that:

1. $r_0 = q_0$ (Initial state)
2. $r_i \in \delta(r_{i-1}, y_i)$ for $i = 1, \dots, m$ (Valid step)
3. $r_m \in F$ (Acceptance on at least one branch)

Existential vs. Universal Logic

The Fundamental Asymmetry:

■ Acceptance is Existential ($\exists$):

$$w \in L(N) \iff \exists \text{ valid path ending in } F$$

If 99 branches get stuck or end in non-accept states, but **1 branch** reaches an accept state in F , the string is **accepted**!

■ Rejection is Universal ($\forall$):

$$w \notin L(N) \iff \forall \text{ paths fail to reach } F$$

A string is rejected only when *every single* computational branch either gets stuck or ends outside F .

Language of an NFA: $L(N) = \{w \in \Sigma^* \mid N \text{ accepts } w\}$. N recognizes exactly one language $L(N)$.

Visualizing NFA Execution: Computation Trees

Tracing parallel state sets and branch prunings step-by-step

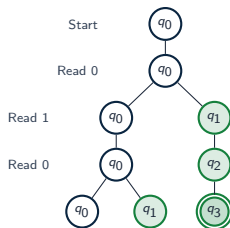
Computation Tree Representation

Parallel Execution Dynamics:

An NFA processing input string $w = 0101$ generates a tree of configurations:

- **Tree Root:** Start state q_0 (plus ε -reachable states).
- **Branching:** On symbol a , each active state splits into $|\delta(q, a)|$ children.
- **Pruning ($\times$):** If $\delta(q, a) = \emptyset$, the branch dies.
- **Acceptance ($\checkmark$):** If any leaf at level $|w|$ has double border $\in F$, string accepts!

Tree Trace for N_1 on $w = 0101$



$q_3 \in F \Rightarrow \text{Accept!}$

Subset Invariant: At any input step k , the NFA is simultaneously in a **subset of active states** $R \subseteq Q$. This subset is the key to converting NFAs into DFAs!

ϵ -Transitions and the ϵ -Closure ($\mathcal{E}(R)$)

Spontaneous zero-cost jumps and computing the reachable state hull

The ϵ -Transition Concept

Spontaneous State Jumps:

An ϵ -transition $\delta(q, \epsilon) = \{p\}$ allows the automaton to jump from q to p **without reading any symbol**.

Key Use Cases:

- Glue sub-automata together modularly.
- Model optional choices (e.g., optional '+' or '-' signs).
- Implement loops without consuming input symbols.

The ϵ -Closure Operator ($\mathcal{E}$)

Definition 3.3: ϵ -Closure $\mathcal{E}(R)$

For any state set $R \subseteq Q$, $\mathcal{E}(R)$ is the set of all states reachable from R by taking **zero or more ϵ -transitions**:

Base: $R \subseteq \mathcal{E}(R)$

Induction: If $q \in \mathcal{E}(R)$ and $p \in \delta(q, \epsilon) \implies p \in \mathcal{E}(R)$

Example Calculation:



$\mathcal{E}(\{1\}) = \{1, 2, 3\}$, $\mathcal{E}(\{2\}) = \{2, 3\}$, $\mathcal{E}(\{3\}) = \{3\}$.

Algorithmic Essence: $\mathcal{E}(R)$ is computed via standard BFS/DFS reachability over ϵ -edges in $\mathcal{O}(|Q| + |E_\epsilon|)$ time.

Equivalence of DFAs and NFAs: The Rabin–Scott Theorem

Michael Sipser, Theorem 1.39: Nondeterminism adds convenience, not computational power

The Fundamental Equivalence Theorem

Theorem 3.1 (Rabin & Scott, 1959):

Every Nondeterministic Finite Automaton (NFA) has an equivalent Deterministic Finite Automaton (DFA) that recognizes the exact same language:

$$L(\text{NFA}) \equiv L(\text{DFA})$$

Corollary: A language L is regular if and only if some NFA recognizes it.

1976 Turing Award Citation:

Michael O. Rabin and Dana Scott received the ACM Turing Award for introducing nondeterminism as a universal conceptual tool in theoretical computer science.

Why This Result is Profound

Contrasting Across the Hierarchy:

Does nondeterminism *always* equal determinism? **No!**

■ Finite Automata (Week 3):

$$\text{DFA} \equiv \text{NFA} \quad (\text{Identical power})$$

■ Pushdown Automata (Week 7):

$$\text{DPDA} \subsetneq \text{NPDA} \quad (\text{Nondeterminism strictly wins!})$$

■ Turing Machines (Week 9):

$$\text{DTM} \equiv \text{NTM} \quad (\text{Same decidable languages})$$

■ Polynomial Time (Week 12):

$$\text{P} \stackrel{?}{=} \text{NP} \quad (\text{\$1M Millennium Prize Problem})$$

The Powerset (Subset) Construction Algorithm

Converting any NFA into an equivalent DFA by simulating active state subsets

Formal Construction Recipe

Given NFA $N = (Q, \Sigma, \delta, q_0, F)$:

Construct equivalent DFA $M = (Q', \Sigma, \delta', q'_0, F')$:

1. **State Space:** $Q' = \mathcal{P}(Q) = 2^Q$.
2. **Start State:** $q'_0 = \mathcal{E}(\{q_0\})$.
3. **Transition Function:** For $R \in Q'$ and $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} \mathcal{E}(\delta(r, a))$$

4. **Accept States:** $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$.

Operational Principles

How the DFA Simulates the NFA:

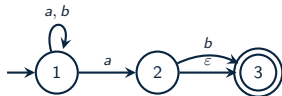
- **Subset Tracking:** At step k , the DFA is in state $R = \{r_1, \dots, r_j\}$, exactly tracking all active NFA parallel branches.
- **The Trap State ($\emptyset$):** The empty set $\emptyset \in Q'$ acts as the dead-end sink state.
- **Lazy Evaluation (Reachability):** While $|Q'| = 2^{|Q|}$ in theory, we only generate states **reachable from q'_0** . In practice, $|Q'_{\text{reachable}}| \ll 2^{|Q|}$.

State Bound: If the NFA has n states, the converted DFA has at most 2^n states.

Step-by-Step Worked Example: Subset Construction

Converting a 3-state NFA with ϵ -transitions into a deterministic machine

Source NFA N_1



Initial State Derivation:

$q'_0 = \mathcal{E}(\{1\}) = \{1\}$.

Note: $\mathcal{E}(\{2\}) = \{2, 3\}$, $\mathcal{E}(\{3\}) = \{3\}$.

Lazy Subset Construction Table

Reachable State Transitions:

DFA State R	Next on a	Next on b	Accept?
$A = \{1\}$	$\{1, 2, 3\}$	$\{1\}$	No
$B = \{1, 2, 3\}$	$\{1, 2, 3\}$	$\{1, 3\}$	Yes ($3 \in F$)
$C = \{1, 3\}$	$\{1, 2, 3\}$	$\{1\}$	Yes ($3 \in F$)

Outcome: Only 3 reachable DFA states $\{A, B, C\}$ out of $2^3 = 8$ potential subsets!

Accept State Rule Applied: States $B = \{1, 2, 3\}$ and $C = \{1, 3\}$ are accept states in the DFA because $3 \in F_{NFA}$.

In-Class Check-in 3.1: NFA Traces & Subset Drill

Diagnostic exercises to test active state tracking and closure logic

Check-in Drill Questions

Problem 3.1A: NFA Trace

Consider the NFA N_1 from Slide 10. Trace the set of active states after reading string $w = aba$:

- Start: $S_0 = \{1\}$.
- What are S_1, S_2, S_3 ? Is aba accepted?

Problem 3.1B: Powerset Intuition

Suppose an NFA has 5 states $Q = \{1, 2, 3, 4, 5\}$ and $F = \{5\}$. Is the DFA subset state $R = \{1, 3, 4\}$ an accepting state?

In-Class Workspace & Discussion

Subset Tracking & Verification:

- **ε -Closure Check:** Always compute $\mathcal{E}(S)$ immediately after computing transition $\delta(S, \sigma)$ to include all states reachable via silent jumps.
- **Parallel State Tracking:** Trace the subset sequence $S_0 \xrightarrow{a} S_1 \xrightarrow{b} S_2 \xrightarrow{a} S_3$.
- **Acceptance Criterion:** Does the terminal set contain at least one state in F ? That is, is $S_3 \cap F \neq \emptyset$?
- **Powerset Logic:** In the converted DFA M' , which subset states are designated as accepting (F')?

[Active learning drill — solutions analyzed interactively in lecture.]

Regular Closure Proofs via NFAs: Concatenation & Star

Michael Sipser, Theorems 1.47 & 1.49: Elegant proofs via ε -gluing

Closure Under Concatenation ($A_1 \circ A_2$)

Theorem 3.2: Concatenation Closure

If A_1 and A_2 are regular, then A_1A_2 is regular.

Construction: Let N_1 recognize A_1 and N_2 recognize A_2 .

- Add an ε -transition from **every accept state of N_1** to the **start state of N_2** .
- Demote N_1 's accept states to non-accept.
- Only N_2 's accept states remain accepting ($F = F_2$).

Closure Under Kleene Star (A_1^*)

Theorem 3.3: Kleene Star Closure

If A_1 is regular, then A_1^* is regular.

Construction: Let N_1 recognize A_1 .

- Add a **new start state** $q_0^{\text{new}} \in F$ with an ε -edge to the old start state of N_1 .
- Add ε -transitions from **every accept state of N_1** back to the old start state.
- *Why new start state?* To accept ε without incorrectly looping non-empty strings!

Algebraic Summary: The 3 regular operations (Union $\cup$, Concatenation $\circ$, Star $*$) all preserve regularity. NFAs make proving this trivially visual!

The State Explosion Problem: Worst-Case 2^n Bound

When deterministic simulation requires exponential memory

The Exponential Lower Bound

Theorem 3.4: DFA State Explosion

For every integer $k \geq 1$, the language:

$$L_k = \{w \in \{0, 1\}^* \mid \text{the } k\text{-th symbol from end is } 1\}$$

can be recognized by an NFA with $k + 1$ states, but **any equivalent DFA requires at least 2^k states**.

Proof Sketch (Pigeonhole Argument): Consider two distinct strings $u \neq v$ of length k . There is some bit position where they differ. Appending a string z accepts for u but rejects for v . Hence, all 2^k prefixes of length k must end in distinct DFA states!

Engineering Scale Comparison

State Space Scaling:

Suffix Dist. (k)	NFA ($k + 1$)	DFA States (2^k)
$k = 4$	5 states	16 states
$k = 8$	9 states	256 states
$k = 16$	17 states	65,536 states
$k = 32$	33 states	$> 4.29 \times 10^9$ (4 GB)

Engineering Dilemma:

Pre-compiling NFAs to DFAs yields $\mathcal{O}(1)$ time per byte but risks out-of-memory crashes on large pattern sets.

Systems Spotlight: Ken Thompson's NFA Engine in Grep

How direct software simulation avoids exponential DFA compilation explosions

The 1968 Thompson Algorithm

Ken Thompson's Breakthrough (CACM 1968):

Instead of converting $\text{regex} \rightarrow \text{NFA} \rightarrow \text{DFA}$ (which can explode to 2^n states), **simulate the NFA directly in software at runtime!**

Runtime Simulation Loop:

1. Maintain a bitset or vector of current active states $S \subseteq Q$.
2. On input character c , compute next state set:

$$S_{\text{next}} = \bigcup_{s \in S} \mathcal{E}(\delta(s, c))$$

3. Repeat for all n characters in the input.

Modern Systems Impact

Algorithmic Guarantees:

- **Time Complexity:** Strictly $\mathcal{O}(m \cdot n)$ where $m = |Q|$ is regex size and n is text length.
- **Space Complexity:** Strictly $\mathcal{O}(m)$ linear memory!
- **No ReDoS Vulnerabilities:** Immune to exponential catastrophic backtracking outages!

Used In Production Today:

Google RE2, Rust regex crate, ripgrep, Linux grep, and Apache Lucene search indexes.

Preview of Week 4: Regular Expressions & GNFA Conversion

Connecting algebraic pattern descriptions with state machine models

Bridging Algebra & Automata

Kleene's Theorem (1956):

A language is regular if and only if it can be described by a **Regular Expression**:

$$\mathcal{L}(\text{DFA}) \equiv \mathcal{L}(\text{NFA}) \equiv \mathcal{L}(\text{Regex})$$

The Three Regular Operations:

1. **Union:** $R_1 \cup R_2$ or $R_1 \mid R_2$.
2. **Concatenation:** $R_1 \circ R_2$ or $R_1 R_2$.
3. **Kleene Star:** R_1^* (zero or more repetitions).

Generalized NFAs (GNFA)



State Elimination Algorithm:

Convert any DFA/NFA into a Regex by placing regexes on transition edges and systematically ripping out states one by one until only q_{start} and q_{accept} remain!

Week 3 Synthesis & Action Items

Mastery checklist, deliverables, and required readings

Week 3 Concept Checklist

You should now be able to:

- Formally specify NFAs as 5-tuples $(Q, \Sigma, \delta, q_0, F)$.
- Trace NFA computation trees and determine existential acceptance $(\exists)$.
- Compute ϵ -closures $\mathcal{E}(R)$ using reachability search.
- Convert any NFA to an equivalent DFA using the Powerset Construction.
- Construct NFAs proving closure under Concatenation $(A \circ B)$ and Star (A^*) .
- Contrast exponential DFA compiling with Thompson's $\mathcal{O}(m \cdot n)$ NFA software simulation.

Deliverables & Upcoming Milestones

Homework & Reading:

- **Problem Set 1:** Submitted today.
- **Required Reading:** Sipser (3rd Ed.), **Section 1.2: Nondeterminism** (pp. 47–58).
- **Upcoming Milestone: Quiz 1** scheduled for Week 4 (covering DFAs, NFAs, and Regex).

Software Practice:

Test your NFAs in **JFLAP** (www.jflap.org) by running step-by-step non-deterministic branching simulations.

Office Hours: Tuesdays & Thursdays 14:00–16:00 GMT | GIMPA School of Technology